

REVERSE

I/P 321

O/P 123

```
int num = sc.nextInt();
```

```
int rev = 0;
```

```
while (num != 0) {
```

```
    int rem = num % 10;
```

```
    rev = rev * 10 + rem;
```

```
    num = num / 10;
```

```
}
```

```
System.out.println(rev);
```

I/P 987

$$\text{rem} = 987 \% 10 = 7$$

$$\text{rev} = 0 * 10 + 7 = 7$$

$$n = 987 / 10 = 98$$

$$\text{rem} = 98 \% 10 = 8$$

$$\text{rev} = 7 * 10 + 8 = 78$$

$$n = 98 / 10 = 9$$

$$\text{rem} = 9 \% 10 = 9$$

$$\text{rev} = 78 * 10 + 9 = 789$$

$$n = 9 / 10 = 0$$

output

989

525

525

12321

PALINDROME

```
int num = sc.nextInt();
```

```
int original = num;
```

```
int rev = 0;
```

```
while (num != 0)
```

```
{
```

```
    int rem = num % 10;
```

```
    rev = rev * 10 + rem;
```

```
    num = num / 10;
```

```
}
```

```
if (original == rev)
```

```
{
```

```
    sopn ("Palindrome");
```

```
}
```

```
else
```

```
{
```

```
    sopn ("Not a Palindrome");
```

```
}
```

PALINDROME

```
int num = sc.nextInt();
```

```
int original = num;
```

```
int rev = 0;
```

```
while (num != 0)
```

```
{
```

```
    int rem = num % 10;
```

```
    rev = rev * 10 + rem;
```

```
    num = num / 10;
```

```
}
```

```
if (original == rev)
```

```
{
```

```
    s o p h (" Palindrome ");
```

```
}
```

```
else
```

```
{
```

```
    s o p h (" Not a Palindrome ");
```

```
}
```

Sum of Given Number

$$i/p = 1\ 2\ 3$$

$$o/p = 1 + 2 + 3 = 6$$

{

int num = sc.nextInt();

int sum = 0;

~~int sum = 0;~~

while (num != 0)

{

int sum = num % 10;

sum = sum + sum;

num = num / 10;

}

System.out.println("sum");

}

$$i/p = 5\ 2\ 1$$

$$o/p = 5 + 2 + 1 = 8$$

$$i/p = 321$$

$$sum = 321 \% 10 = 1$$

$$sum = 0 + 1 = 1$$

$$num = 321 / 10 = 32$$

$$sum = 32 \% 10 = 2$$

$$sum = 1 + 2 = 3$$

$$num = 32 / 10 = 3$$

$$sum = 3 \% 10 = 3$$

$$sum = 3 + 3 = 6$$

$$num = 3 / 10 = 0$$

$$o/p =$$

6

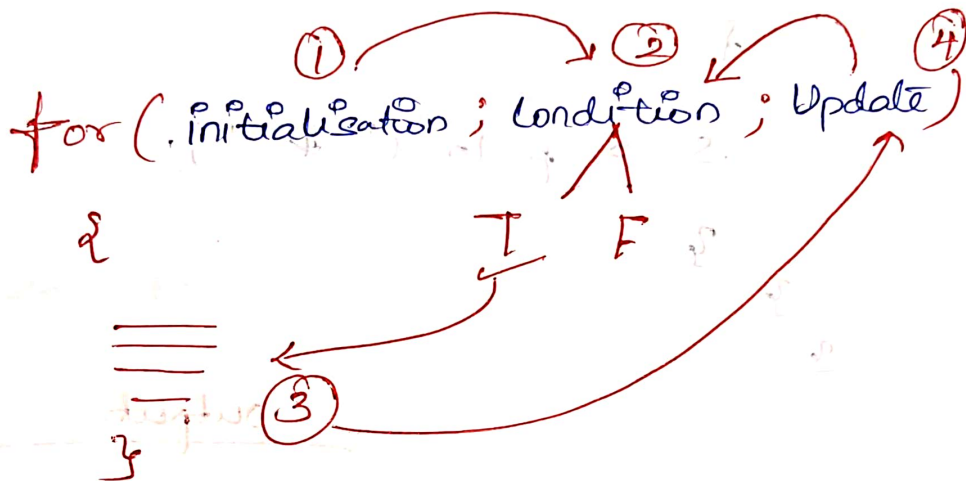
25/07/2025

FOR LOOP

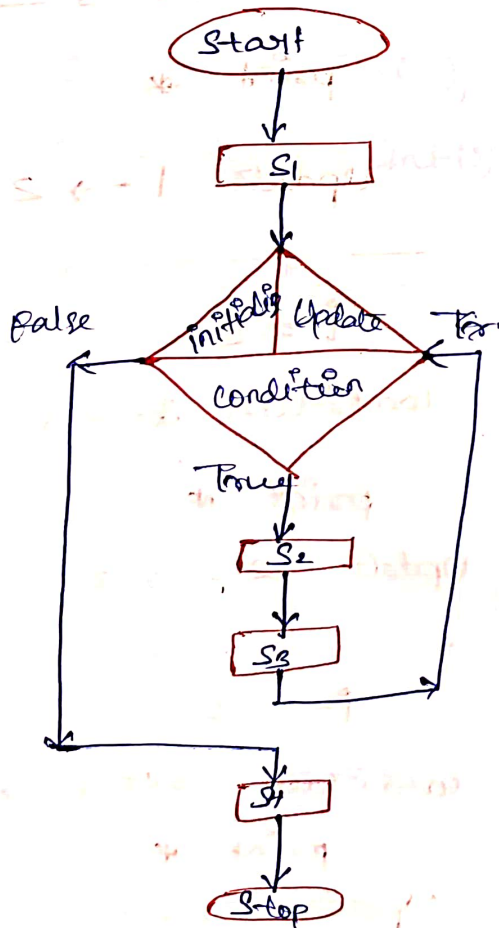
→ It is a keyword in Java

⇒ If we know the N no. of iteration the best loop is 'FOR'

Syntax



Flow chart



Eg:-

class Demo {

public static void main (String[] args)

{

for (int i = 1 ; i <= 3 ; i++)

{

System.out.println ("*");

}

}

}

Output

int i = 1

condition $1 \leq 3 \rightarrow T$

print *

Update $1 \rightarrow 2$

i = 2

condition $2 \leq 3 \rightarrow T$

print *

Update $2 \rightarrow 3$

i = 3

condition $3 \leq 3 \rightarrow T$

print *

Update $3 \rightarrow 4$

i = 4 condition false

O/P

*
*
*

Q) Find power of given Number.

int b = sc.nextInt(); // base Value

int e = sc.nextInt(); // Exponential Value

int pow = 1;

$$2^3 = 8$$

for (int i = 1; i <= e; i++)

2 → base

3 → Exponential

{

pow = pow * b; pow = pow * b;

}

System.out.println(pow);

}

Q) ~~Even~~ Even number in the range.

int s = sc.nextInt(); // start Value

int e = sc.nextInt(); // End Value

for (i = s; i <= e; i++)

{

if (i % 2 == 0)

{

System.out.println(i);

}

}

26/7/25

medium moving for moving 1000 1000

Q)

Need output as

1 2 3 4 5
2 3 4 5 6
3 4 5 6 7
4 5 6 7 8
5 6 7 8 9

int n = sc.nextInt();

for (int i = 1; i <= n; i++)

{
for (int j = 1; j <= n; j++)

sc.print(" "); // cursor on same line

3 sopln(); cursor breaks and get to next line

1 2 3 4 5 6 7 8 9 10

1 2 3 4 5 6 7 8 9 10

1 2 3 4 5 6 7 8 9 10

1 2 3 4 5 6 7 8 9 10

1 2 3 4 5 6 7 8 9 10

1

28/07/2025

Output

i

01	02	03	04	05
06	07	08	09	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

scanf int n = &nextInt();

int count = 1;

for (int i = 1; i <= n; i++) {
 for (int j = 1; j <= n; j++) {

if (count <= 9) {
 }

so print(0);
 }

so print(count++ + " ");

}

so.println();

}

}

i = 1
 j = 1
 count = 1 <= 9

count 0
 1 2 3 4 5 6 7 8 9

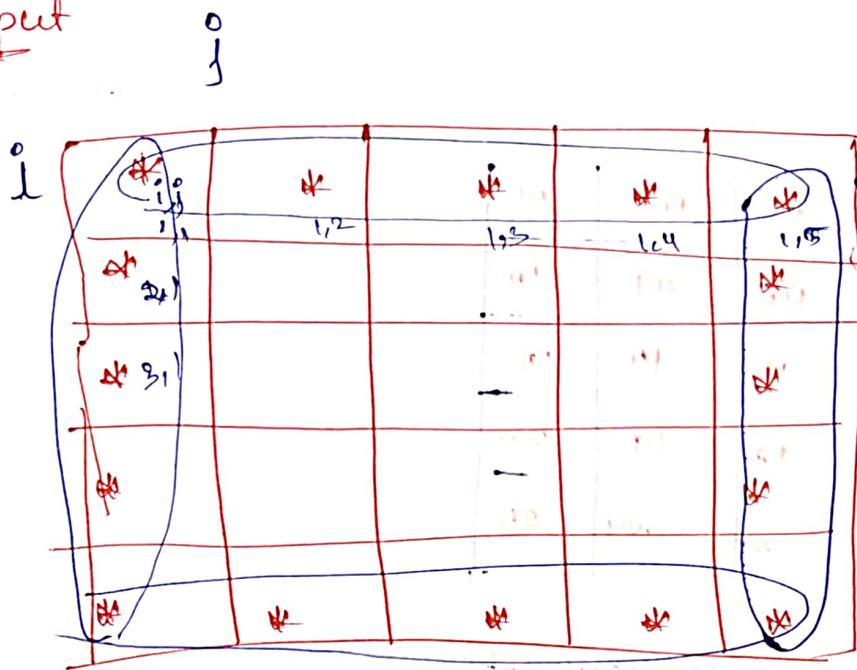
i = 1

i	j	
1	1	1 <= 5 → T
2	2	2 <= 5 → T
3	3	3 <= 5 → T
4	4	4 <= 5 → T
5	5	5 <= 5 → T
6	6	6 <= 5 → F

j = 4

01	02	03	04	05
06	07	08	09	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

Output



$i = 1$
 $j = 1$
 $i = 5$
 $j = 5$

```
int n = sc.nextInt();
```

```
for (int i = 1; i <= n; i++)
```

```
{
    for (int j = 1; j <= n; j++)
```

```
{
```

```
if (i == 1 || j == 1 || i == n || j == n)
```

```
{
    s.o.print("@");
```

```
}
```

```
else
```

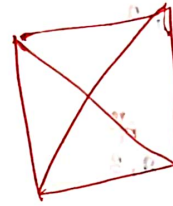
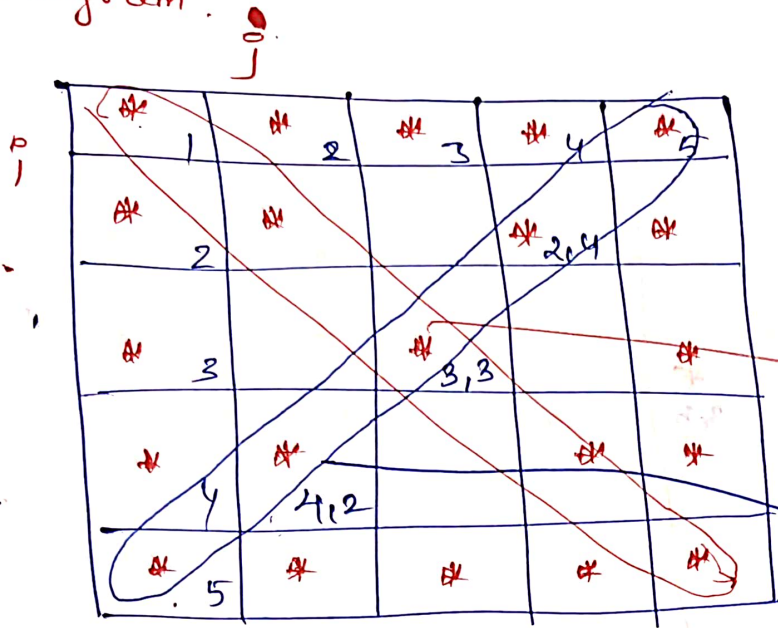
```
{
```

```
s.o.print(" "); // two space.
```

```
}
```

```
s.o.println();
```

Program.



Primary diagonal ($i=j$)

Secondary diagonal ($i+j=6$)

```
for (int i = 1; i <= 5; i++)
```

```
{
```

```
    for (int j = 1; j <= 5; j++)
```

```
        if (i == 1 || j == 1 || i == 5 || j == 5 ||
```

```
            (i == j || i + j == 6))
```

```
        {
```

```
            cout << " * ";
```

```
        }
```

```
    }
```

```
    }
```

```
    cout << endl;
```

```
}
```

```
main()
```

```
{
```

Program -

		*		
		1, 3		
		*		
		2, 3		
*	*	@	*	*
3, 1	3, 2	3, 3	3, 4	3, 5
		*		
		4, 3		
		*		
		5, 3		

```
int n = 5
```

```
for (int i = 1; i <= n; i++)
```

```
{
```

```
    for (int j = 1; j <= n; j++)
```

```
    {
```

```
        if (i == 3 || j == 3)
```

```
        {
```

```
            if (i == 3 && j == 3)
```

```
            {
```

```
                s.o. print("@ ");
```

```
            }
```

```
        else
```

```
        {
```

```
            s.o. print(" * ");
```

```
        }
```

```
    else {
```

```
        s.o. print(" ");
```

```
    }
```

```
    s.o. println();
```

Program.

* → 1 star
 * * → 2 stars
 * * * → 3 stars
 * * * * → 4 stars
 * * * * * → 5 stars

```
int n = sc.nextInt();
for (int i = 1; i <= n; i++)
```

```
{
    for (int j = 1; j <= i; j++)
```

```
{
    s.o. print("*");
```

```
    s.o. println();
```

```
int n = sc.nextInt();
```

```
int star = 1;
```

```
for (int i = 1; i <= n; i++)
```

```
{
    for (int j = 1; j <= star; j++)
```

```
{
```

```
    s.o.p("*");
```

```
}
```

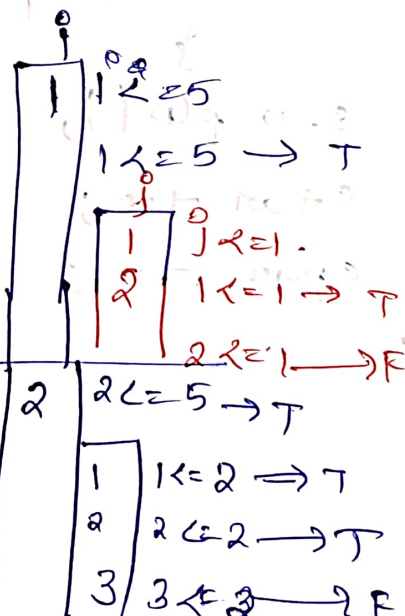
```
    s.o.println();
```

```
    star++;
```

}

star

1 2 3



Program

```

1
  *
 * *
* * *
 * * *
* * * *

```

Space	Star
4	1
3	2
2	3
1	4
0	5

```
int n = sc.nextInt();
```

```
int star = 1;
```

```
int space = n - 1;
```

```
for (int i = 1; i <= n; i++)
```

```
{
    for (int j = 1; j <= space; j++)
```

```
{
```

```
        s.o.p(" ");
```

```
    }
```

```
    for (int k = 1; k <= star; k++)
```

```
{
```

```
        s.o.p("* ");
```

```
    }
```

```
    s.o.pln();
```

```
    star++;
```

```
    space--;
```

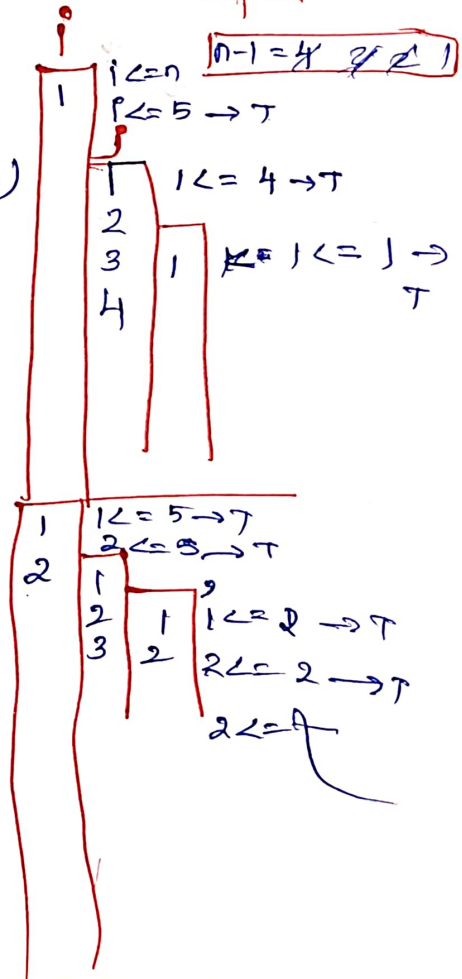
```
}
```

star

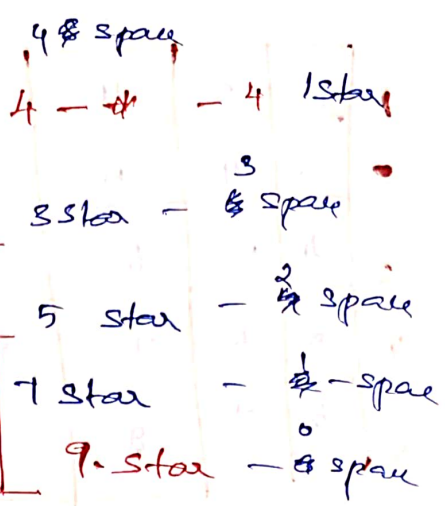
~~1 2 3 4 5~~

Space

~~n-1=4 3 2 1~~



expositio



0 — space, 9 — star

432

5) $5 \leftarrow 4 \rightarrow 12 \rightarrow 12$

4 123

спра - -

Pattern.

1	2	3	4	5	6
2	3	4	5	6	7
3	4	5	6	7	8
4	5	6	7	8	9
5	6	7	8	9	10
	1	2	3	4	5

~~i+j~~

```
int n = sc.nextInt();
```

```
for (int i = 1; i <= n; i++)
```

```
{
```

```
    for (int j = 1; j <= n; j++)
```

```
    {
```

```
        s.o.p(i+j + " ");
```

```
    }
```

```
    s.o.pln();
```

```
}
```

altern.

A	3	4	5	6
2	B	5	6	7
4	5	C	7	8
5	6	7	D	9
6	7	8	9	E

$i = j$

1	2	3	4	5
2	3	4	5	6
3	4	5	6	7
4	5	6	7	8
5	6	7	8	9

```
int n = sc.nextInt();
```

```
char c = 'A';
```

```
for (int i = 1; i <= n; i++)
```

```
{
```

```
    for (int j = 1; j <= n; j++)
```

```
    {
```

```
        if (i == j)
```

```
        {
```

```
            if
```

```
            s.o.p(c++ + " - " + "A");
```

```
        }
```

```
    } else
```

```
    {
```

```
        s.o.p(i + j);
```

```
    }
```

```
    s.o.println();
```

```
}
```

Pattern

1	2	3	4	5
2	3	4	5	6
3	4	5	6	7
4	5	6	7	8
5	6	7	8	9



int n = sc.nextInt();

for (int i = 1 ; i <= n ; i++)

{

for (int j = 1 ; j <= n ; j++)

{

s.o.p(i+j-1);

}

s.o.pn();

}

Pattern -

*	*	*	*	*	*	*	*	*
	*	*	*	*	*	*	*	
	*	*	*	*	*	*		
			*	*	*			
			*	*				

space

0
1
2
3
4

star

9
7
5
3
1

```
int n = sc.nextInt();
```

```
int star = n * 2 - 1;
```

```
int space = 0;
```

```
for (int i = 0; i <= n; i++)
```

```
{
    for (int j = 0; j <= space; j++)
```

```
{
    s.o.p(" ");
```

```
}
```

```
for (int k = 1; k <= star; k++)
```

```
{
```

```
    s.o.p(" * ");
```

```
}
```

```
star = star - 2;
```

```
space ++;
```

```
s.o.ph();
```

```
}
```

1. 2 3 4 5

3, 2

4, 2 4, 3

5, 2 5, 3 5, 4

5

१७३-॥ ६.

$$p + j\omega + 1$$
$$f(x) = 0 \quad \forall x \in \mathbb{R}$$

2

 ~~$9 \quad (i^0 + i^0 = 5 \quad || \quad i^0 + i^0 = 6 \quad || \quad j^0 + i^0 = 57)$~~

Armstrong Number.

$$153 = (1)^3 + (5)^3 + (3)^3$$

$$1634 = (1)^4 + (6)^4 + (3)^4 + (4)^4$$

```
import java.util.Scanner;  
class Armstrong {
```

```
Scanner sc = new Scanner(System.in)
```

```
{  
    int n = sc.nextInt();
```

```
    int dc = count(n);
```

```
    if (isArmstrong(n, dc))
```

```
    {
```

```
        s.o.pln("Armstrong");
```

```
    }
```

```
    else
```

```
    {
```

```
        s.o.pln("Not an Armstrong");
```

```
    }
```

```
    public static int count(int num)
```

```
    {
```

```
        int count = 0;
```

```
        while (num != 0)
```

```
        {
```

```
            num = num/10;
```

```
            count++;
```

```
        }
```

```
        return count;
```

```
    }
```

```

public static boolean isArmstrong (int num, int count)
{
    int sum = 0;
    int ori = num;
    while (num != 0)
    {
        int rem = num % 10;
        sum = sum + pow (rem, count);
        num = num / 10;
    }
    if (ori == sum)
    {
        return true;
    }
    else
    {
        return false;
    }
}

public static int pow (int base, int exp)
{
    int pow = 1;
    for (int i = 1; i <= exp; i++)
    {
        pow = pow * base;
    }
    return pow;
}

```

Factors of Num.

```
int n = s.next Int();
```

```
s.o.pln("Enter a number");
```

```
int n =
```

```
printFact(n);
```

```
p s v printFact(int num)
```

```
{
```

```
int count = 0;
```

```
for (int i = 1; i <= num; i++)
```

```
{
```

```
if (num % i == 0)
```

```
{
```

```
s.o.pln(i);
```

```
count++;
```

```
}
```

```
}
```

```
s.o.pln("Total num of factors " + " " + count);
```

```
}
```

```
}
```

Perfect Number

if p 6

1 2 3 6
↓ fact

```
class PerfectNumber {
```

```
    public static void main (String [] arg) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();
```

```
        isPerfectNumber(n)
```

```
    }
```

```
    public static void isPerfectNumber (int n)
```

```
    {
```

```
        for (int i = 1; i < n; i++)
```

```
        {
```

```
            if (n % i == 0)
```

```
            {
```

```
                System.out.println ("Factors are " + i);
```

```
                sum = sum + i;
```

```
            }
```

```
        }  
        if (sum == n)
```

```
        {
```

```
            System.out.println ("Perfect Num");
```

```
        }  
        else
```

```
        {  
            System.out.println ("Not Perfect Num");
```

```
        }
```

```
    }
```

1 + 2 + 3 = 6 ✓

Perfect Number

Factorial

5!

$$1 \times 2 \times 3 \times 4 \times 5$$

int n = sc.nextInt();

isFact(n);

}

public static void isFact(int n)

{

int fact = 1;

for (int i = 1; i <= n; i++)

{

fact = fact * i;

sc.println("The factorial is " + fact);

}

}

1 2 6 24 120

5

i i <= n
1 1 <= 5 → T

fact = 1 * 1 = 1

2 2 <= 5 → T

fact = 1 * 2 = 2

3 3 <= 5 → T

fact = 2 * 3 = 6

4 4 <= 5 → T

fact = 6 * 4 = 24

5 5 <= 5 → T

fact = 24 * 5 = 120

Strong Number

i/p 145

$$1! = 1$$

$$4! = 24$$

$$5! = 120$$

$$145$$

Strong Number

```
int n = sc.nextInt();
```

```
int sum = 0;
```

```
int ori = n;
```

```
while (n != 0)
```

```
{
```

```
    int rem = n % 10;
```

```
    sum = sum + fact(rem);
```

```
    n = n / 10;
```

```
}
```

```
if (ori == sum)
```

```
{
```

```
    s.o.pl ("Strong Num");
```

```
}
```

```
else {
```

```
    s.o.pl ("Not a strong Num");
```

```
}
```

```
}
```

n 145

while

rem

5 4

sum

0 120 144

$$rem = 145 / 10 = 5$$

$$sum = 0 + 120$$

$$n = 145 / 10 = 14$$

$$sum = 14 * 10 = 140$$

$$sum = 120 + 24$$

$$n = 14 / 10 = 1$$

$$rem = 1 * 10 = 1$$

$$sum = 144 + 1$$

$$n = 1 / 10 = 0$$

$$0! = 1 \Rightarrow \text{True}$$

public static int fact (int num)

{

int fact = 1;

for (int i = 1; i <= num; i++)

{

fact = fact * i;

}

return fact;

}

1 2 6 24 120

fact

i

1

1 <= 5 → T

1 * 1 = 1

2

2 <= 5 → T

1 * 2 = 2

3

3 <= 5 → T

2 * 2 = 6

4

4 * 6 = 24

5

5 * 24 = 120

04/08/2025

Happy Number

```
int num = sc.nextInt();
int sum = 0;
while (n != 0 || sum > 9)
{
    if (n == 0)
    {
        n = sum;
        sum = 0;
    }
    int rem = n % 10;
    sum = sum + rem * rem;
    n = n / 10;
}
s.o.pln(sum);
if (sum == 1)
{
    s.o.pln("Happy Num");
}
else
{
    s.o.pln("Not happy Num");
}
```

13 + 10
1 + 10 = 6
1/10

$$13 = 1^2 + 3^2 = 10$$

$$10 = 1^2 + 0^2 = 1$$

$$19 = 1^2 + 9^2 = 82$$

$$8^2 + 2^2 = 64 + 4 = 70$$

$$7^2 + 0^2 = 49 + 0 = 49$$

finally come to 1

while
n 19 82 70 49 13 10 1
sum 0 81 82 0 4 68 64 100 1

while (n != 0 || sum > 9)

19 != 0 || 0 > 9 - F
↓
T

19 == 0 - F

sum = 19 % 10 = 9

sum = 0 + 9 * 9 = 81

n = 19 / 10 = 1

1 != 0 || 81 > 9 - F
↓
T

1 == 0 - F

sum = 1 % 10 = 1

sum = 81 + 1 * 1 = 82

n = 1 / 10 = 0

$$0 \neq 0 \parallel 82 > 9 \rightarrow T$$

↓
F

$$if (0 == 0) - T$$

$$n = 82$$

$$sum = 0;$$

$$rem = 82 \% 10 = 2$$

$$sum = 0 + 2 * 2 = 4$$

$$n = 82 / 10 = 8$$

$$8 \neq 0 \parallel 4 > 9 - F$$

↓
T

$$8 == 0 \rightarrow F$$

$$rem = 8 \% 10 = 8$$

$$sum = 4 + 8 * 8 = 68$$

$$n = 8 / 10 = 0$$

$$\text{Finally } sum == 1$$

$$0 \neq 0 \parallel 1 > 9 \rightarrow F$$

↓
F

$$\& \text{ o. pln}(sum)$$

$$1 \neq 0 \parallel 0 > 9$$

$$rem = 1 \% 10 = 1$$

$$sum = 0 + 1 * 1 = 1$$

$$n = 1 / 10 = 0$$

$$0 \neq 0 \parallel 1 > 9 \rightarrow F$$

$$0 \neq 0 \parallel 68 > 9$$

$$0 == 0 \rightarrow T$$

$$n = 68$$

$$sum = 0$$

$$rem = 68 \% 10 = 8$$

$$sum = 0 + 8 * 8 = 64$$

$$n = 68 / 10 = 6$$

$$6 \neq 0 \parallel 64 > 9$$

$$if \rightarrow F$$

$$rem = 6 \% 10 = 6$$

$$sum = 64 + 6 * 6 = 100$$

$$n = 6 / 10 = 0$$

$$0 \neq 0 \parallel 100 > 9$$

$$if 0 == 0 \rightarrow T$$

$$n = 100$$

$$sum = 0$$

$$rem = 100 \% 10 = 0$$

$$sum = 0 + 0 * 0 = 0$$

$$n = 100 / 10 = 10$$

$$10 \neq 0 \parallel 0 > 9$$

↓
F

↓
F

$$rem = 10 \% 10 = 0$$

$$sum = 0 + 0 * 0 = 0$$

$$n = 10 / 10 = 1$$

Automorphic Number

$$5^2 = \underline{25}$$

$$6^2 = \underline{36}$$

$$76^2 = \underline{5776}$$

$$376^2 = \underline{141376}$$

Automorphic Number.

```
int n = sc.nextInt();
int ori = n;
int div = 1;
while (n != 0)
{
    n = n/10;
    div = div * 10;
}
int pow = ori * ori;
int rem = pow % div;
if (rem == ori)
{
    s.o. pr ("AN");
}
else
{
    s.o. pr ("Not an AN");
}
```

while $n \neq 0$

n 7670

div ~~1~~ 10 100

pow 5776

ori 76

rem 76

$76 \neq 0 \rightarrow T$

$n = 76/10 = 7$

$div = 1 * 10 = 10$

~~$pow =$~~

$7 \neq 0 \rightarrow T$

$n = 7/10 = 0$

$div = 10 * 10 = 100$

$pow = 76 * 76 = 5776$

$rem = 5776 \% 100$
 $= 76$

$76 == 76$
 $\text{if } rem == ori \rightarrow T$

~~s.o. pr~~ $76 == 76$
 $\downarrow$
 T

$print ("AN");$